

```

import math

# =====
# CSA S16 Chapter 4 - Flexural Members
# CIVE 3205 - Design of Structural Steel Components
# =====
# Covers:
#   - Shear strength (Cl. 13.4) - unstiffened and stiffened webs
#   - Section classification Table 2 (flexural compression)
#   - Laterally supported members (Cl. 13.5)
#   - Laterally unsupported members (Cl. 13.6)
#   - Deflection limits (Table D.1)
# =====

PHI = 0.9 # resistance factor for steel

# =====
# SECTION 1: SHEAR STRENGTH (Cl. 13.4)
# =====

def shear_yield_stress_unstiffened(h, w, Fy):
    """
    Compute shear yield stress  $F_s$  for unstiffened webs (Cl. 13.4.1.1).

    Parameters
    -----
    h : float - clear depth of web (mm)
    w : float - web thickness (mm)
    Fy : float - yield strength (MPa)

    Returns
    -----
    dict with  $F_s$  (MPa) and governing case label
    """
    ratio = h / w
    sqrt_Fy = math.sqrt(Fy)

    lim1 = 1014.0 / sqrt_Fy
    lim2 = 1435.0 / sqrt_Fy

    if ratio <= lim1:
        Fs = 0.66 * Fy
        case = "i)  $h/w \leq 1014/\sqrt{F_y} \Rightarrow F_s = 0.66F_y$ "

```

```

elif ratio <= lim2:
    Fs = 670.0 * sqrt_Fy / ratio
    case = "ii) 1014 < h/w <= 1435 => Fs = 670*sqrt(Fy)/(h/w)"
else:
    Fs = 961200.0 / (ratio ** 2)
    case = "iii) h/w > 1435/sqrt(Fy) => Fs = 961200/(h/w)^2"

return {"Fs": Fs, "case": case, "h_over_w": ratio}

def shear_buckling_coefficients(a, h):
    """
    Compute kv and ka for stiffened webs.

    Parameters
    -----
    a : float - distance between stiffeners (mm)
    h : float - web depth (mm)

    Returns
    -----
    dict with kv, ka
    """
    aspect = a / h
    if aspect < 1.0:
        kv = 4.0 + 5.34 / (aspect ** 2)
    else:
        kv = 5.34 + 4.0 / (aspect ** 2)

    ka = 1.0 / math.sqrt(1.0 + aspect ** 2)
    return {"kv": kv, "ka": ka, "a_over_h": aspect}

def shear_yield_stress_stiffened(h, w, Fy, a):
    """
    Compute shear yield stress Fs for stiffened webs (Cl. 13.4.1.1b).

    Parameters
    -----
    h : float - web depth (mm)
    w : float - web thickness (mm)
    Fy : float - yield strength (MPa)
    a : float - stiffener spacing (mm)

    Returns
    -----
    dict with Fs, Fcri, Fcre, kv, ka, and case label

```

```

"""
coeff = shear_buckling_coefficients(a, h)
kv = coeff["kv"]
ka = coeff["ka"]

sqrt_Fy = math.sqrt(Fy)
ratio = h / w

Fcri = 290.0 * math.sqrt(Fy * kv) / ratio
Fcre = 180000.0 * kv / (ratio ** 2)

lim1 = 439.0 * math.sqrt(kv / Fy)
lim2 = 502.0 * math.sqrt(kv / Fy)
lim3 = 621.0 * math.sqrt(kv / Fy)

if ratio <= lim1:
    Fs = 0.66 * Fy
    case = "i) h/w <= 439*sqrt(kv/Fy) => Fs = 0.66Fy"
elif ratio <= lim2:
    Fs = Fcri
    case = "ii) <= 502*sqrt(kv/Fy) => Fs = Fcri"
elif ratio <= lim3:
    Fs = Fcri + ka * (0.50 * Fy - 0.866 * Fcri)
    case = "iii) <= 621*sqrt(kv/Fy) => Fs = Fcri + ka*(0.5Fy - 0.866Fcri)"
else:
    Fs = Fcre + ka * (0.50 * Fy - 0.866 * Fcre)
    case = "iv) h/w > 621*sqrt(kv/Fy) => Fs = Fcre + ka*(0.5Fy - 0.866Fcre)"

return {
    "Fs": Fs, "case": case,
    "Fcri": Fcri, "Fcre": Fcre,
    "kv": kv, "ka": ka,
    "h_over_w": ratio
}

def factored_shear_resistance(h, w, Fy, a=None):
    """
    Compute factored shear resistance  $V_r = \phi * A_w * F_s$ .

    Parameters
    -----
    h : float - web depth (mm)
    w : float - web thickness (mm)
    Fy : float - yield strength (MPa)
    a : float or None - stiffener spacing; None => unstiffened

```

```

Returns
-----
dict with Vr (kN), Fs, Aw, and governing case
"""
Aw = h * w # mm^2
if a is None:
    result = shear_yield_stress_unstiffened(h, w, Fy)
else:
    result = shear_yield_stress_stiffened(h, w, Fy, a)

Fs = result["Fs"]
Vr = PHI * Aw * Fs / 1000.0 # convert N to kN
return {
    "Vr_kN": Vr,
    "Fs_MPa": Fs,
    "Aw_mm2": Aw,
    "shear_details": result
}

# =====
# SECTION 2: TABLE 2 SECTION CLASSIFICATION (Flexural)
# =====

def classify_flange_one_edge(bel, t, Fy):
    """
    Table 2: Element supported along one edge under flexural compression.
    Applies to: flanges of I/T-sections, outstanding angle legs.

    Returns class (1, 2, 3, or 4) and limiting ratios.
    """
    ratio = bel / t
    sqrt_Fy = math.sqrt(Fy)
    lim1 = 145.0 / sqrt_Fy
    lim2 = 170.0 / sqrt_Fy
    lim3 = 200.0 / sqrt_Fy

    if ratio <= lim1:
        cls = 1
    elif ratio <= lim2:
        cls = 2
    elif ratio <= lim3:
        cls = 3
    else:
        cls = 4

    return {

```

```

        "class": cls, "bel_over_t": ratio,
        "lim_class1": lim1, "lim_class2": lim2, "lim_class3": lim3
    }

```

```

def classify_stem_tee(bel, t, Fy):

```

```

    """

```

```

    Table 2: Element supported along one edge with part in tension.

```

```

    Applies to: stems of T-sections, I-section flange under minor-axis bending.

```

```

    Class 3 limit is  $340/\sqrt{F_y}$  instead of  $200/\sqrt{F_y}$ .

```

```

    """

```

```

    ratio = bel / t

```

```

    sqrt_Fy = math.sqrt(Fy)

```

```

    lim1 = 145.0 / sqrt_Fy

```

```

    lim2 = 170.0 / sqrt_Fy

```

```

    lim3 = 340.0 / sqrt_Fy

```

```

    if ratio <= lim1:

```

```

        cls = 1

```

```

    elif ratio <= lim2:

```

```

        cls = 2

```

```

    elif ratio <= lim3:

```

```

        cls = 3

```

```

    else:

```

```

        cls = 4

```

```

    return {

```

```

        "class": cls, "bel_over_t": ratio,

```

```

        "lim_class1": lim1, "lim_class2": lim2, "lim_class3": lim3

```

```

    }

```

```

def classify_rhs_flange(bel, t, Fy):

```

```

    """

```

```

    Table 2: RHS flanges (two edges supported) under flexural bending.

```

```

    """

```

```

    ratio = bel / t

```

```

    sqrt_Fy = math.sqrt(Fy)

```

```

    lim1 = 420.0 / sqrt_Fy

```

```

    lim2 = 525.0 / sqrt_Fy

```

```

    lim3 = 670.0 / sqrt_Fy

```

```

    if ratio <= lim1:

```

```

        cls = 1

```

```

    elif ratio <= lim2:

```

```

        cls = 2

```

```

elif ratio <= lim3:
    cls = 3
else:
    cls = 4

return {
    "class": cls, "bel_over_t": ratio,
    "lim_class1": lim1, "lim_class2": lim2, "lim_class3": lim3
}

def classify_box_flange(bel, t, Fy):
    """
    Table 2: Box section flanges (two edges supported).
    Class 1 and Class 2 share the same limit (525/sqrt(Fy)).
    """
    ratio = bel / t
    sqrt_Fy = math.sqrt(Fy)
    lim1 = 525.0 / sqrt_Fy
    lim2 = 525.0 / sqrt_Fy
    lim3 = 670.0 / sqrt_Fy

    if ratio <= lim1:
        cls = 1
    elif ratio <= lim3:
        cls = 3
    else:
        cls = 4

    return {
        "class": cls, "bel_over_t": ratio,
        "lim_class1": lim1, "lim_class2": lim2, "lim_class3": lim3
    }

def classify_web_combined(h, w, Fy, Cf, Cy):
    """
    Table 2: Web of I-section under combined flexure + axial compression.

    Parameters
    -----
    h : float - clear web depth (mm)
    w : float - web thickness (mm)
    Fy : float - yield strength (MPa)
    Cf : float - factored axial compression (kN)
    Cy : float - squash load = Ag * Fy (kN)

```

```

Returns
-----
dict with class and limits
"""
ratio = h / w
sqrt_Fy = math.sqrt(Fy)
phi_Cy = PHI * Cy
cf_ratio = Cf / phi_Cy

lim1 = (1100.0 / sqrt_Fy) * (1.0 - 0.39 * cf_ratio)
lim2 = (1700.0 / sqrt_Fy) * (1.0 - 0.61 * cf_ratio)
lim3 = (1900.0 / sqrt_Fy) * (1.0 - 0.65 * cf_ratio)

if ratio <= lim1:
    cls = 1
elif ratio <= lim2:
    cls = 2
elif ratio <= lim3:
    cls = 3
else:
    cls = 4

return {
    "class": cls, "h_over_w": ratio,
    "lim_class1": lim1, "lim_class2": lim2, "lim_class3": lim3
}

def classify_chs(D, t, Fy):
    """
    Table 2: Circular hollow sections.

    Parameters
    -----
    D : float - outer diameter (mm)
    t : float - wall thickness (mm)
    Fy: float - yield strength (MPa)
    """
    ratio = D / t
    lim1 = 13000.0 / Fy
    lim2 = 18000.0 / Fy
    lim3 = 66000.0 / Fy

    if ratio <= lim1:
        cls = 1
    elif ratio <= lim2:
        cls = 2

```

```

elif ratio <= lim3:
    cls = 3
else:
    cls = 4

return {
    "class": cls, "D_over_t": ratio,
    "lim_class1": lim1, "lim_class2": lim2, "lim_class3": lim3
}

# =====
# SECTION 3: LATERALLY SUPPORTED MEMBERS (Cl. 13.5)
# =====

def moment_resistance_laterally_supported(section_class, Z, S, Fy, Se=None):
    """
    Factored moment resistance for laterally supported members (Cl. 13.5).

    Parameters
    -----
    section_class : int    - 1, 2, 3, or 4
    Z              : float  - plastic section modulus (mm^3)
    S              : float  - elastic section modulus (mm^3)
    Fy             : float  - yield strength (MPa)
    Se             : float or None - effective section modulus for Class 4 web cas

    Returns
    -----
    dict with Mr (kN*m), Mp, My, and case
    """
    Mp = Z * Fy / 1e6 # kN*m
    My = S * Fy / 1e6 # kN*m

    if section_class in (1, 2):
        Mr = PHI * Mp
        case = "Class 1/2: Mr = phi*Z*Fy = phi*Mp"
    elif section_class == 3:
        Mr = PHI * My
        case = "Class 3: Mr = phi*S*Fy = phi*My"
    else:
        # Class 4
        if Se is not None:
            Mr = PHI * Se * Fy / 1e6
            case = "Class 4 (web OK, flange Class4): Mr = phi*Se*Fy"
        else:
            # Conservative: use My, caller should handle S136 if needed

```

```

        Mr = PHI * My
        case = "Class 4: Mr = phi*My (conservative; check CSA S136 if both ex

    return {"Mr_kNm": Mr, "Mp_kNm": Mp, "My_kNm": My, "case": case}

# =====
# SECTION 4: Laterally Unsupported Members (Cl. 13.6)
# =====

def omega2_general(M_max, M_a, M_b, M_c):
    """
    Equivalent moment factor omega2 for general moment distribution (Eq. 2).
    All moments are absolute values in consistent units.

    M_max : maximum moment in unbraced segment
    M_a    : moment at quarter-point
    M_b    : moment at mid-point
    M_c    : moment at three-quarter point
    """
    denom = math.sqrt(M_max**2 + 4*M_a**2 + 7*M_b**2 + 4*M_c**2)
    w2 = 4.0 * M_max / denom
    return min(w2, 2.5)

def omega2_linear(k):
    """
    Equivalent moment factor omega2 for linear moment gradient (Eq. 3).

    k : ratio of smaller to larger end moment
        positive for double curvature, negative for single curvature
    """
    w2 = 1.75 + 1.05 * k + 0.3 * k**2
    return min(w2, 2.5)

def critical_elastic_moment(L, E, Iy, G, J, Cw, omega2):
    """
    Critical elastic lateral-torsional buckling moment Mu (Eq. 1, Cl. 13.6).

    Parameters
    -----
    L      : float - unbraced length (mm)
    E      : float - elastic modulus (MPa), typically 200000
    Iy     : float - weak-axis moment of inertia (mm^4)
    G      : float - shear modulus (MPa), typically 77000
    J      : float - St. Venant torsional constant (mm^4)

```

```

Cw      : float - warping constant (mm^6); use 0 for HSS
omega2   : float - equivalent moment factor

Returns
-----
Mu in kN*m
"""
term1 = E * Iy * G * J
term2 = (math.pi * E / L) ** 2 * Iy * Cw
Mu = (omega2 * math.pi / L) * math.sqrt(term1 + term2)
return Mu / 1e6 # N*mm -> kN*m

def moment_resistance_laterally_unsupported(
    section_class, Mu_kNm, Mp_kNm, My_kNm
):
    """
    Factored moment resistance Mr for laterally unsupported members (Cl. 13.6).

    Parameters
    -----
    section_class : int - 1, 2, 3, or 4
    Mu_kNm        : float - critical elastic moment (kN*m)
    Mp_kNm        : float - plastic moment capacity (kN*m)
    My_kNm        : float - yield moment capacity (kN*m)

    Returns
    -----
    dict with Mr (kN*m) and governing case description
    """
    if section_class in (1, 2):
        M_ref = Mp_kNm
        label = "Mp"
        threshold = 0.67 * M_ref
        if Mu_kNm > threshold:
            Mr = 1.15 * PHI * M_ref * (1.0 - 0.28 * M_ref / Mu_kNm)
            Mr = min(Mr, PHI * M_ref)
            case = "Class 1/2, Mu > 0.67Mp: Mr = 1.15*phi*Mp*(1 - 0.28*Mp/Mu) "
        else:
            Mr = PHI * Mu_kNm
            case = "Class 1/2, Mu <= 0.67Mp: Mr = phi*Mu"
    else:
        # Class 3 or 4
        M_ref = My_kNm
        label = "My"
        threshold = 0.67 * M_ref
        if Mu_kNm > threshold:

```

```

        Mr = 1.15 * PHI * M_ref * (1.0 - 0.28 * M_ref / Mu_kNm)
        Mr = min(Mr, PHI * M_ref)
        case = "Class 3/4, Mu > 0.67My: Mr = 1.15*phi*My*(1 - 0.28*My/Mu) "
    else:
        Mr = PHI * Mu_kNm
        case = "Class 3/4, Mu <= 0.67My: Mr = phi*Mu"

    return {
        "Mr_kNm": Mr,
        "Mu_kNm": Mu_kNm,
        "M_ref_kNm": M_ref,
        "threshold_kNm": threshold,
        "case": case
    }

# =====
# SECTION 5: DEFLECTION LIMITS (Table D.1)
# =====

DEFLECTION_LIMITS = {
    # Industrial - Vertical
    "industrial_floor": ("L/300", 300),
    "industrial_inelastic_roof": ("L/240", 240),
    "industrial_elastic_roof": ("L/180", 180),
    "crane_girder_heavy": ("L/800", 800),    # >= 225 kN
    "crane_girder_light": ("L/600", 600),    # < 225 kN
    # Industrial - Lateral
    "crane_lateral": ("L/600", 600),
    # All others - Vertical
    "other_floor_crack_susceptible": ("L/360", 360),
    "other_floor_no_crack": ("L/300", 300),
    # All others - Lateral
    "wind_drift_building": ("h/400", 400),
    "storey_drift_cladding": ("h/500", 500),
    "storey_drift_special": ("h/400", 400),
}

def max_deflection(span_or_height_mm, application_key):
    """
    Compute maximum allowable deflection for a given application.

    Parameters
    -----
    span_or_height_mm : float - span L or storey height h (mm)
    application_key    : str   - key from DEFLECTION_LIMITS dict

```

```

Returns
-----

dict with delta_max (mm), limit string, and divisor
"""

if application_key not in DEFLECTION_LIMITS:
    raise ValueError(
        f"Unknown application key '{application_key}'. "
        f"Valid keys: {list(DEFLECTION_LIMITS.keys())}"
    )
label, divisor = DEFLECTION_LIMITS[application_key]
delta_max = span_or_height_mm / divisor
return {
    "delta_max_mm": delta_max,
    "limit_label": label,
    "divisor": divisor
}

# =====
# CONVENIENCE: FULL BEAM DESIGN CHECK
# =====

def beam_design_check(
    section_class, Z, S, Fy,
    Mf_kNm,
    L_mm, E, Iy, G, J, Cw,
    M_max, M_a, M_b, M_c,
    h_web, w_web,
    Vf_kN,
    laterally_supported=True,
    a_stiffener=None,
    Se=None
):
    """
    Full beam design check: shear, section classification assumed done,
    moment resistance (laterally supported or unsupported), and unity checks.

    Returns a results dict for display/reporting.
    """
    # --- Shear ---
    shear = factored_shear_resistance(h_web, w_web, Fy, a=a_stiffener)
    shear_ok = Vf_kN <= shear["Vr_kN"]

    # --- Moment ---
    if laterally_supported:
        moment = moment_resistance_laterally_supported(

```

```

        section_class, Z, S, Fy, Se=Se
    )
    Mr = moment["Mr_kNm"]
else:
    omega2 = omega2_general(M_max, M_a, M_b, M_c)
    Mu = critical_elastic_moment(L_mm, E, Iy, G, J, Cw, omega2)
    Mp = Z * Fy / 1e6
    My = S * Fy / 1e6
    moment = moment_resistance_laterally_unsupported(
        section_class, Mu, Mp, My
    )
    Mr = moment["Mr_kNm"]

moment_ok = Mf_kNm <= Mr

return {
    "shear": shear,
    "shear_ok": shear_ok,
    "Vf_kN": Vf_kN,
    "moment": moment,
    "moment_ok": moment_ok,
    "Mf_kNm": Mf_kNm,
    "Mr_kNm": Mr,
    "section_class": section_class,
}

```